

Inhaltsverzeichnis

1. Syntax	1
2. Anweisungen (Statements)	2
3. Schlüsselwörter	3

1. Syntax

Die **Syntax** definiert Aufbau und Form von Programmen:

- Lexikalische Elemente: **Identifiers** (z.B. Variablen), **Literale** (Zahlen, Zeichenketten) & **Operatoren**.
 - Identifier: Namen für Variablen, Klassen, Methoden. In Java beginnen sie mit Buchstaben, `_` oder `$` und dürfen danach Ziffern enthalten.
 - Literale: feste Werte wie Zahlen, Zeichenketten, `true/false`, `null`, `42`, `"Hallo"`.
 - Operatoren: Symbole zur Operation auf Werten, z.B. `+`, `-`, `*`, `/`, `==`, `&&`, `||`.
- **Kommentare**: Einzeilige (`//...`) und mehrzeilige Kommentare (`/* ... */`) zur Dokumentation.
- Die Grundstruktur von Java-Programmen sind (die Summe aller implementierten) Deklarationen von
 - Paketen,
 - Klassen,
 - Methoden und
 - Algorithmen

Wichtige **Regeln**:

- Eine Java Klasse wird in einer Datei mit der Endung `.java` gespeichert, der Dateiname entspricht dem Klassennamen, das kompilierte Pendant hat die Endung `.class` (Plattformunabhängigkeit: Der Quellcode wird in Bytecode kompiliert, der auf jeder JVM ausgeführt werden kann)
- Java unterscheidet streng zwischen Groß- und Kleinschreibung,
- Variablen sollen einen wohlüberlegten, ausdrucksstarken Namen haben
- Codeblöcke werden durch `{ ... }` abgegrenzt, vor allem Klassen und Methoden
- Jede Anweisung endet mit einem Semikolon `...;`
- Klassennamen beginnen mit *Großbuchstaben*, Methodennamen mit *Kleinbuchstaben*. Bei der Schreibweise wird explizit die Einhaltung der gängigen Praxis, des sogenannten "CamelCase" empfohlen
- Variablen haben einen Datentyp, er sollte angegeben werden, muss er aber nicht

Beispiel:

```
// Paketdeklaration
package de.dhbw;

// import-Anweisungen (Paketstrukturen werden durch "." getrennt,
// sie entsprechen Verzeichnissen)
import java.util.List;

// Klassensignatur
// Schlüsselwörter + Klassenname (mit Großbuchstaben beginnend)
public class MyFirstExample { // Klassenkörper mit Codeblöcken

    // Methodensignatur
    // Schlüsselwörter + Methodenname (mit Kleinbuchstaben beginnend)
    // Methodenparameter: Variablendeklarationen in runden Klammern
    public int sumOfNumbers(int a, int b) {
        // Schlüsselwort mit Anweisung, abgeschlossen mit Semikolon
        return a + b;
    } // Ende der Methode

} // Ende der Klasse
```

2. Anweisungen (Statements)

Statements sind ausführbare Anweisungen, die wie bereits gesagt mit einem Semikolon abgeschlossen werden. Dazu zählen:

- Deklarationen (z.B. Datentyp und Variable deklarieren, `String`, `var`)

```
// Variablen-Deklaration mit explizitem Typ
int count;
// Variablen-Deklaration mit Typinferenz
var count;
```

- Ausdrucks-Statements (z.B. Methodenaufruf, Zuweisung)

```
// Zuweisung ist ein Ausdrucks-Statement
int a = 5;

// Methodenaufruf als Statement
System.out.println("Hallo Welt!");
```

- Kontrollflussanweisungen:
 - Bedingte Anweisungen: `if`, `else`, `switch`
 - Schleifen: `for`, `while`, `do-while`
- Sprunganweisungen: `return`, `break`, `continue`, `throw`

- Blockanweisungen (Codeblöcke): *geschweifte Klammern* gruppieren mehrere Statements, also {
... }

```
...  
public void blockExample() {  
    int temp = 42;  
    System.out.println("Temp: " + temp);  
}  
...
```

3. Schlüsselwörter

Java kennt eine Vielzahl von vordefinierten Anweisungen, den sogenannten Schlüsselwörtern. Dies sind *reservierte* Wörter mit spezieller Bedeutung. Beispiele (Auswahl):

- `class`, `interface`, `enum`
- `public`, `private`, `protected`, `static`, `final`
- `if`, `else`, `switch`, `case`
- `for`, `while`, `do`, `break`, `continue`
- `return`, `new`, `this`, `super`, `null`, `true`, `false`